

Buy or Wait? — AI Financial Decision Agent

Comprehensive Technical & System Architecture Q&A; for Interviews & AI Judge

Author: Solo Participant | Challenge: HackerRank Orchestrate (September 2026) | Language: Hinglish

Q1

Project Title & One-line Summary

Kis problem ko solve karta hai?

- Project Title: Buy or Wait? — Autonomous AI Financial Decision & Cash-Flow Advisor.
- One-line Summary: Ek intelligent, conservative financial decision agent jo user ke live bank balance, recurring EMIs, fixed living expenses, aur salary cycles ko 90 din tak simulate karke accurately batata hai ki koi purchase abhi karna chahiye, installment/EMI par lena chahiye, salary tak wait karna chahiye, ya postpone/cancel karna chahiye.

Q2

Why did you build it? (Real-world Motivation & Pain Point)

Is project ki ahem zaroorat kyun mehsoos hui?

- Aggressive Marketing & 1-Click Checkout: Aaj kal e-commerce platforms par No-Cost EMI, Credit Cards, aur BNPL (Buy Now Pay Later) ki wajah se impulsive spending bahut aasan ho gayi hai.
- Real-World Pain Point: Log apna current account balance dekh kar impulsively shopping kar lete hain, lekin agle 15-20 dino mein aane wale fixed kharche (Rent, Ration, Bijli ka bill, Bachchon ki school fees) aur Emergency Fund ko bhool jaate hain.
- The Result: Month-end par cash crunch aur high-interest debt trap shuru ho jata hai. Is agent ko checkout button dabane se pehle ek unbiased, rational financial guardian ki tarah act karne ke liye build kiya gaya hai.

Q3

Problem Statement (Existing Solutions ki Khamiyan)

Existing apps aur models mein kya kami thi?

- Static Post-Facto Tracking: Existing expense trackers (jaise Walnut, Mint) sirf kharcha hone ke baad pie-chart dikhate hain. Woh checkout se pehle real-time decision-making mein guidance nahi dete.
- Pure LLMs ki Hallucination: Generic generative LLMs (GPT-4/Claude) basic arithmetic, calendar dates, aur multi-currency exchange rates mein severe hallucination karte hain, jo financial decisions ke liye dangerous hai.
- Zero Safety-Buffer Logic: Standard affordability calculators sirf (Balance - Price) dekhte hain, bina yeh samjhe ki Emergency Fund aur upcoming essential commitments ko kabhi touch nahi karna hota.

Q4

Your Solution (End-to-End Architecture & Deliverables)

Tumne end-to-end kya build kiya?

- Deterministic 90-Day Simulation Engine: Day-by-day cash flow projection algorithm jo pending debits, confirmed salary settlement, aur essential needs ko track karta hai.
- 4-Tier Affordability Classifier: Har request ko 4 standards mein categorize karta hai: `affordable_now`, `affordable_with_plan`, `affordable_later`, ya `not_affordable`.
- Multi-Option Payment Planner: Seller options aur user preferences ko match karke Full Payment, Partial Schedule, ya No-Cost EMI plans generate karta hai.
- Interactive Bilingual Web Application: Flask + Bootstrap 5 dashboard jisme 250 evaluation cases ka explorer aur naye users ke liye custom budget simulator (English & Hindi) integrated hai.
- HackerRank Orchestrate Submission: 250 test requests ka fully validated `output.csv` aur \$0.00 token cost ka `usage_report.md`.

Q5

Tech Stack Justification

Specific languages, tools aur architecture kyun choose kiye?

- Python 3.11: Financial data processing, date arithmetic (`datetime`), aur array logic ke liye maximum reliability aur speed.
- Deterministic Algorithm vs Pure LLM: Financial math mein 100% deterministic simulation engine use kiya. Isse 0 API latency, \$0.00 token cost, aur zero arithmetic hallucination achieve hui. Har benchmark calculation mathematically verifiable aur reproducible hai.
- Flask + Bootstrap 5: Heavy React/Node setup ke badle lightweight Flask use kiya jo zero-config standalone executable hai aur kisi bhi browser par instantly load hota hai.
- Relational In-Memory Mapping: CSV datasets ko indexed hash-maps mein load kiya gaya, jisse evaluation ke waqt $O(1)$ instant lookup milta hai.

Q6

Your Exact Role & Contributions

Is project mein tumhara specific role kya tha?

- Solo Architect & Full-Stack AI Engineer: HackerRank Orchestrate challenge rules ke mutabiq yeh 100% author-only solo submission hai.
- Core Engine Design: 90-day cash balance simulator, conservative foreign currency conversion logic, aur multi-tier payment planner ka mathematical model develop kiya.
- Guardrail Implementation: Strict constraint filters implement kiye — unrealized stock gains drop kiye, pending debits reserve kiye, aur protected family support categories ko preserve kiya.
- Product & UI Engineering: Live bilingual dashboard, comprehensive user guide (`USER_GUIDE.md`), aur automated evaluation pipeline develop karke GitHub par deploy kiya.

Q7

Architecture & Data Flow

Frontend se Core Engine tak request kaise travel karti hai?

- 1. Request Ingestion: Frontend Form ya Evaluation CSV se Purchase Request (Amount, Date, Currency, Options) trigger hoti hai.
- 2. User Context Loading: Engine user ka Financial Profile, Current Available Balance, Minimum Reserve, aur Scheduled Events load karta hai.
- 3. FX & Cash Filter: Foreign transactions ko settlement date-matched rates se convert kiya jata hai; unrealized investments ko drop aur pending debits ko block kiya jata hai.
- 4. 90-Day Simulation Engine: Har aane wale din ($t=1$ to 90) par check hota hai: $\text{Balance}[t] \geq \text{Minimum_Balance}$. Agar kisi bhi din reserve breach hua toh flag raise hota hai.
- 5. Decision Arbiter: Best safe payment method (Full, EMI, Wait, ya Postpone) choose hota hai aur grounded explanation ke sath output format mein deliver hota hai.

Q8

Database Schema & APIs

Primary entities, relationships aur API endpoints kya hain?

- Primary Entities:
 - financial_profiles: user_id (PK), current_available_balance, minimum_balance_to_keep, expense_categories_to_protect, max_installment_months.
 - financial_events: event_id (PK), user_id (FK), event_type, cash_impact_status (settled, pending, scheduled, unrealized), settlement_date, amount.
 - requests: request_id (PK), user_id (FK), requested_amount, request_date, desired_completion_date.
 - exchange_rates: date, from_currency, to_currency, rate.
- Key API Endpoints:
 - GET /: Responsive bilingual dashboard (supports ?lang=en|hi, ?tab=custom|eval|guide|faq).
 - POST /simulate: Real-time dynamic simulator calculation for custom user budgets.
 - GET /api/evaluate/<request_id>: Programmatic JSON API returning evaluation metrics.

Q9

Major Technical Challenges & Resolution

Koi bugs, latency issues ya logic edge cases kaise resolve kiye?

- Challenge 1: Non-Cash & Unrealized Assets: Dataset mein stocks aur mutual funds ke unrealized gains the jo cash balance ko artificially inflate kar rahe the. Resolution: Strict cash-state validator lagaya jo sirf settled liquid cash aur confirmed salary ko count karta hai.
- Challenge 2: macOS Port 5000 AirPlay Conflict: macOS par AirPlay Receiver port 5000 par occupy rehta hai. Resolution: Server ko dynamically host='0.0.0.0', port=8080 par configure kiya aur robust daemon lifecycle add ki.
- Challenge 3: Untrusted Evidence & Prompt Injection: Messages aur images mein fake cancellation ya altered discount claims the. Resolution: Strict rule hierarchy enforce ki — explicit settlement and bank ledger facts always override user text messages.

Q10

Future Enhancements (Scalability & Cloud Integration)

Is system ko aage scale aur expand kaise karenge?

- Account Aggregator (AA) Integration: RBI ke Sahamati framework ke zariye direct live bank account consent integration taaki statements real-time auto-sync ho sakein.
- UPI 2.0 & AutoPay Mandates: NPCI ke recurring mandate APIs integrate karna taaki monthly SIPs, OTT bills, aur EMIs automatically simulation calendar mein map ho sakein.
- Multimodal Receipt OCR: Physical receipts aur salary slips scan karke instant budget entry ke liye lightweight vision models (e.g. Gemini Nano / Tesseract) link karna.
- Cloud Microservice Deployment: Dockerize karke AWS ECS / GCP Cloud Run par deploy karna with Redis session store for high-throughput enterprise scale.